Pact tooling as part of this book is because Pact provides you with a mock provider easily.

4.1.3 Using loose matchers to avoid high coupling

Another general recommendation when writing consumer contract tests is to be aware of the loose matchers concept to avoid high coupling. High coupling occurs when a change in one area affects other areas. Let's think of coupling in the context of LEGO blocks. In a *high coupling scenario*, the LEGO blocks are tightly interlocked, and removing or changing one block can cause the entire structure to collapse. These blocks depend heavily on each other to stay together, making it challenging to modify or maintain the structure. Imagine trying to replace one block in a tightly woven LEGO wall—it's difficult without disrupting the whole thing. On the other hand, in a *low coupling scenario*, each LEGO block is self-contained and doesn't rely heavily on other blocks to function. You can easily assemble and disassemble your structure because the blocks are loosely connected. In contract testing, high coupling occurs when a small change from the provider, even if it's just the data it returns, causes the consumer contract test to fail. This is one reason why a contract test might be flaky and brittle and, if you're not careful, will suffer the same fate as a flaky integration test. For more context on this, check out Martin Fowler's "Tolerant Reader" blog article (https://martinfowler.com/bliki/TolerantReader.xhtml).

To combat flaky tests, tools such as Pact provide *loose matchers*, a functionality that allows for more flexible matching of data, to ensure that consumers verify the shape of the interaction rather than the actual data it returns. Loose matchers can be in the form of type-based matching, arrays, or even plain regular expressions. With type-based matching, you care more about the type of data a specific field returns, rather than the actual data itself. Here's an example code snippet:

```
const EXPECTED_BODY = {
  id: like(1),
  username: like('marie'),
  fullname: like('marie cruz')
}
```